MADRAS INSTITUTE OF TECHNOLOGY
Department of Information Technology

B.Tech. Artificial Intelligence and Data Science

Subject: Machine Learning Operations

PROJECT REPORT

CHAINCOMMAND

An Intelligent Supply Chain Risk, Procurement Optimization and Inventory Management Platform

Submitted by

Vignesh B – 2023510043
Srimathi S – 2023510055

Anna University

23 October 2026

1. Title

ChainCommand: An Intelligent Supply Chain Risk, Procurement Optimization, and Inventory Management Platform

2. Aim

To develop and demonstrate a supply-chain operations platform that combines machine learning, bill-of-materials (BOM) analysis, reinforcement learning, supplier-risk scoring, constraint optimization and Clear-to-Build (CTB) checks. The platform aims to help assess demand, evaluate risk, plan inventory and procurement, identify material shortages, and monitor operational KPIs.

3. Project Team Members

1. Vignesh B – 2023510043
2. Srimathi S – 2023510055

4. Techniques Used

- **Demand forecasting:** LSTM and XGBoost forecasts combined using product-specific ensemble weights.
- **Anomaly detection:** a trained detector flags unusual patterns in generated supply-chain data.
- **Supplier-risk assessment:** a machine-learning risk model and risk-scoring component estimate supplier risk.
- **Bill of Materials (BOM) analysis:** assembly component trees are generated and their roll-up costs are calculated.
- **Reinforcement learning:** a Proximal Policy Optimization (PPO) method is trained for inventory-related decisions.
- **Constraint optimization:** CP-SAT is used for procurement/supplier allocation under configured constraints.
- **Clear-to-Build (CTB):** required components are compared with availability to detect shortages.
- **KPI and event monitoring:** inventory and fulfillment metrics are computed and threshold violations are published as events.

5. Tools Used

- **Language/runtime:** Python 3.11 and a Python virtual environment.
- **Forecasting/ML:** LSTM, XGBoost and machine-learning anomaly/risk components.
- **Reinforcement learning:** PPO.
- **Optimization:** CP-SAT constraint solver.
- **Application modules:** ChainCommand Python package and asynchronous event bus.
- **Development tools:** Windows PowerShell and Git.
- **Evidence:** the command-line demo `python -m chaincommand --demo`.

6. Architecture

The demo log shows a pipeline in the order ML → BOM → RL → Risk → CP-SAT → CTB. The ML stage trains forecasting and anomaly models; the BOM stage creates assembly component trees; RL trains a PPO policy; the risk stage trains a supplier-risk model; CP-SAT handles constrained allocation; and CTB checks whether assemblies have enough materials. A KPI engine calculates summary metrics and emits events when thresholds are violated.

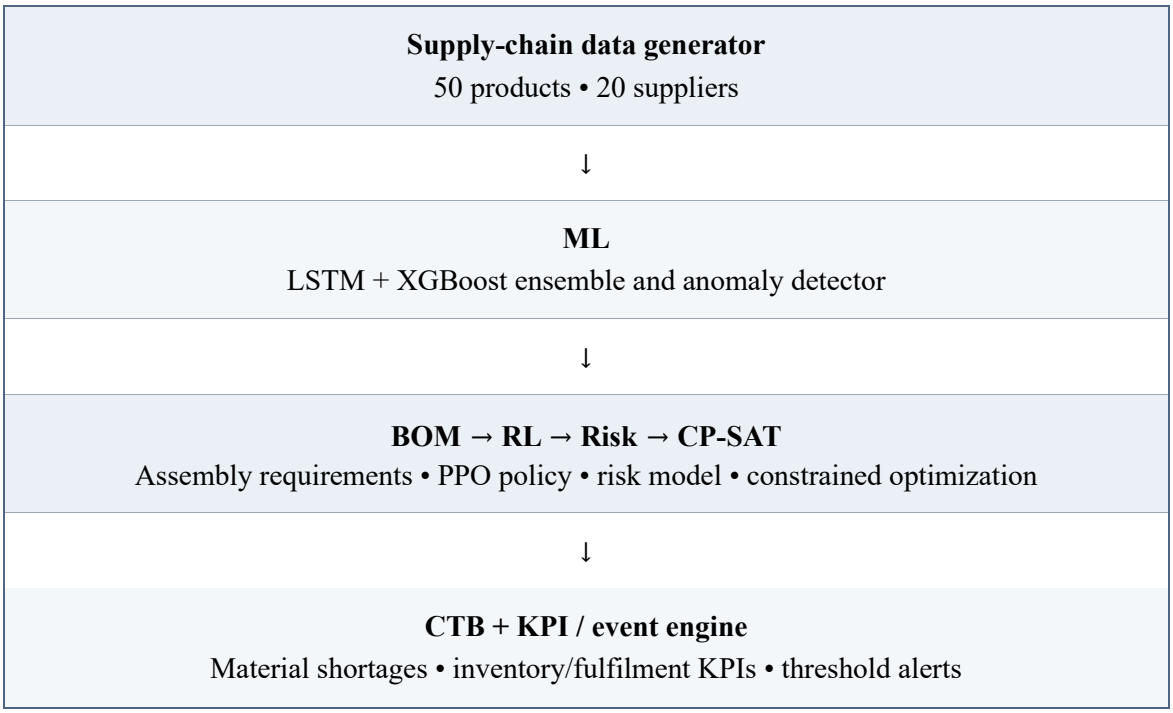


Figure 1. Architecture reconstructed from the observed demo workflow.

7. Methodology

1. Initialize the application and generate synthetic supply-chain data.

2. Train the LSTM and XGBoost forecasting models for product demand and compute ensemble weights.

3. Train the anomaly detector across the generated product set.

4. Generate BOM trees for five assemblies and compute component roll-up costs.

5. Train the PPO reinforcement-learning policy for 50,000 timesteps.

6. Train the supplier-risk model and initialize the risk-scoring component.

7. Compute the initial KPI snapshot and start optimization cycle 1.

8. Run CTB analysis for assemblies, record component shortages, update KPIs, and publish anythreshold-violation events.

9. Shut down the event bus and monitoring process after the demo cycle.

8. Output Expected

- Generated dataset containing 50 products and 20 suppliers.
- Trained LSTM and XGBoost models with ensemble weights for product demand forecasting.
- Trained anomaly detector and supplier-risk model.
- Five generated BOM assemblies with component counts and roll-up cost.
- PPO training summary and risk-model evaluation summary.
- Inventory KPI snapshot including days of supply (DSI), fill rate, OTIF and stockouts.
- CTB results indicating whether assemblies can be built and identifying material shortages.
- Events for KPI threshold violations.

Observed demo values: 50 products, 20 suppliers; five BOM assemblies; 55 total BOM items; BOM roll-up cost

1270.86; risk-model accuracy 0.70 on 20 test samples; initial KPI snapshot DSI 20.5, fill rate 1.0, OTIF 0.0 and stockouts 0. CTB reported clear_pct 0.0 and shortages of 9, 7 and 8 for assemblies ASM-0001, ASM-0002 and ASM-0003 respectively. The PPO training log reported a negative mean reward and improvement_pct of -272.52%, which suggests the RL policy requires further investigation rather than demonstrating successful policy improvement.

9. Output Screenshot

The following screenshots were captured from the actual local run of `python -m chaincommand --demo`. They document model training, BOM initialization, PPO and risk-model training, KPI computation, CTB shortage analysis, and the final exception.

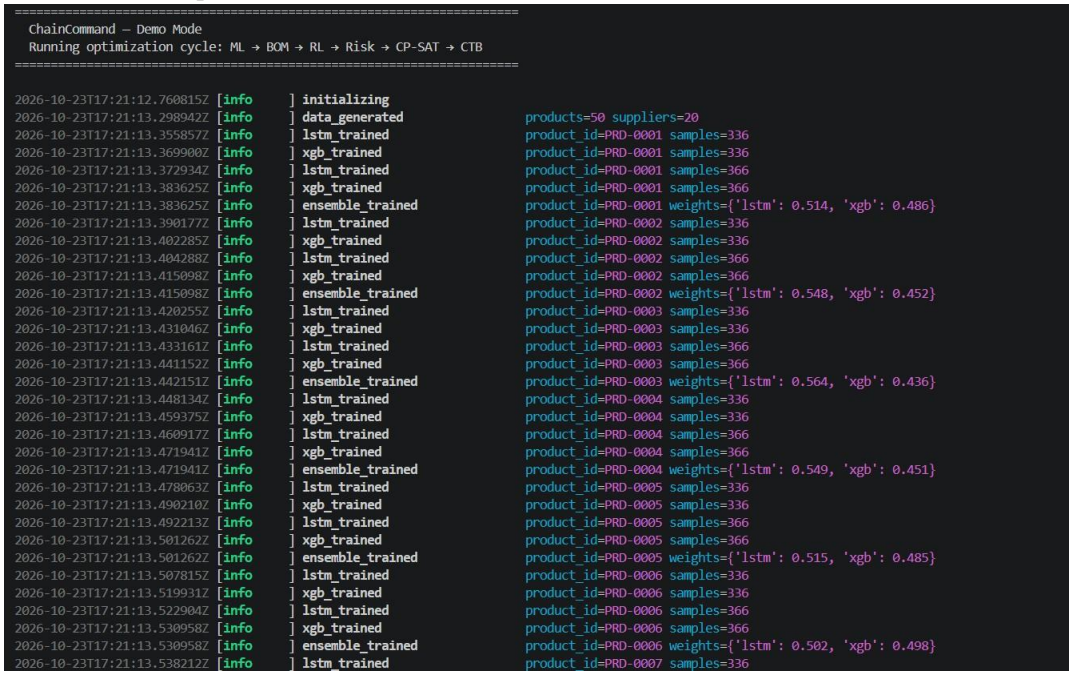


Figure 1. Actual demo output screenshot (1 of 4).

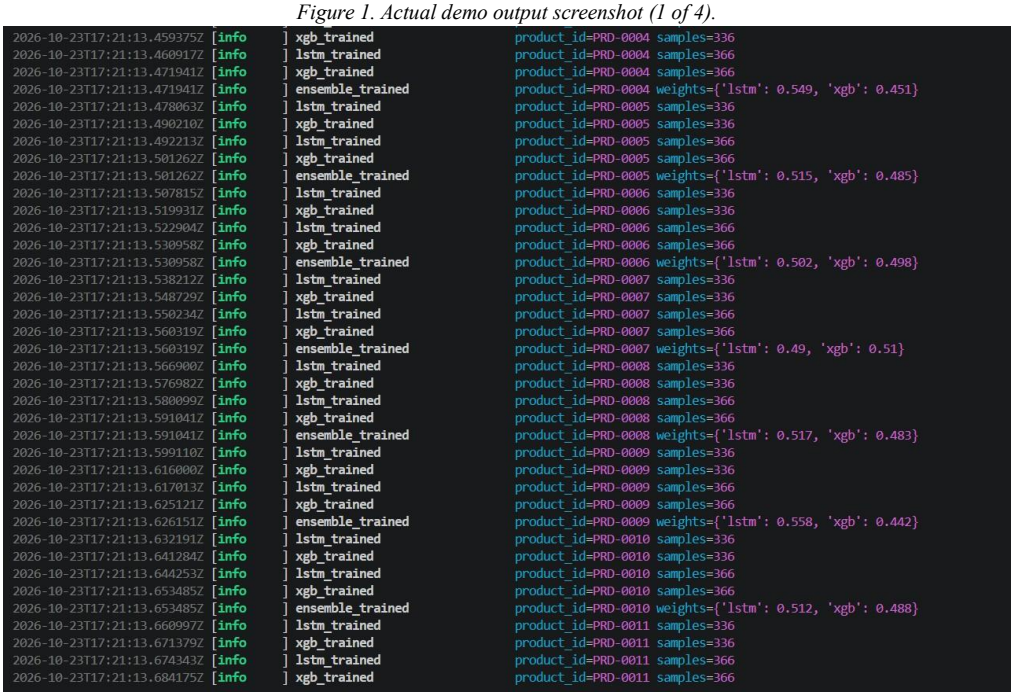


Figure 2. Actual demo output screenshot (2 of 4).

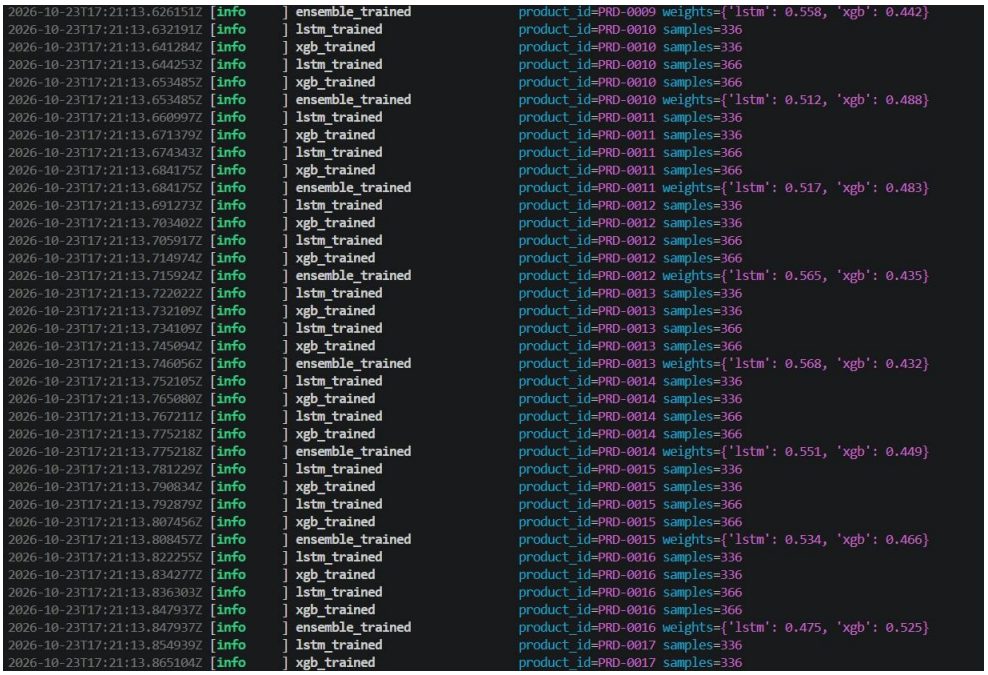


Figure 3. Actual demo output screenshot (3 of 4).

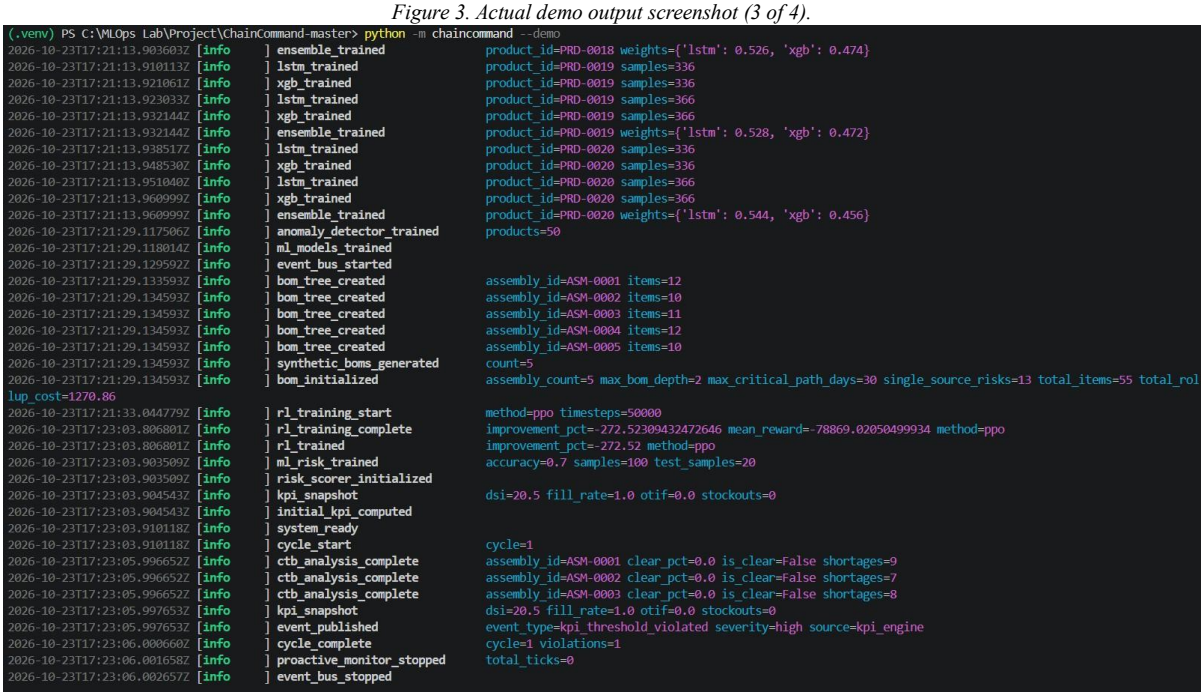


Figure 4. Actual demo output screenshot (4 of 4).

10. Conclusion

The observed run demonstrates that ChainCommand can generate synthetic supply-chain data, train demand-forecasting and anomaly/risk models, build BOM trees, train a PPO policy, compute KPIs and perform CTB shortage checks. The output also exposes areas requiring improvement: the PPO run reports negative reward/improvement, CTB identifies significant shortages in the generated assemblies, and the demo result-printing code crashes because the returned result dictionary lacks the `cycle` key expected by `__main__.py`. The project is therefore a working prototype with a substantial end-to-end workflow, but its RL performance, shortage outcomes and final demo exception should be addressed and retested before production use.

Appendix: Reproducibility

Command executed in Windows PowerShell from `C:\MLOps Lab\Project\ChainCommand-master`:
`python -m chaincommand --demo`

The log warns that a default API key is used in development. A real secret should be configured through `CC_API_KEY` before any deployment. The run timestamp in the logs is 23 October 2026.